Amaranta

Estudiante	Brandow Stivent Claros Polania Y Victor Manuel Brand Cepeda
Proyecto	Amaranta - Tienda y Clinica Veterinaria de Mascotas
Sprint	Sprint 4 (Semana 4 del corte) - Sesion 1
Duracion	3 horas de trabajo en equipo
Cadencia	Un sprint cada 8 dias, acordado en equipo desde el Sprint 1
Tema visto en clase esa semana	Construir un servicio: estructura por capas y el walking skeleton
Fecha	31 de agosto de 2026
Repositorio	github.com/BrandowC/amaranta-shop-docs

1. Que buscabamos en este sprint

El objetivo de esta sesion era pasar del diseno (las reglas de negocio que en el Sprint 3 protegimos dentro de Orden y Cita) a un servicio que de verdad corra: organizar el codigo por capas (dominio / aplicacion / adaptadores), conectar las piezas concretas en un unico punto del arranque, y construir un "walking skeleton" - el corte mas delgado que funciona de punta a punta - para tener siempre algo demostrable en vez de solo documentos.

2. Que avanzamos en el proyecto

- Elegimos el area de Ordenes (Ventas) como el primer servicio para construir el walking skeleton -ordenes-service-, dejando Citas para el siguiente sprint, ya que ambas son areas nucleo pero no alcanzan las dos a la vez.
- Creamos la carpeta del servicio por capas: domain/ (Orden, objetos de valor y los puertos como interfaces), application/ (el caso de uso ConfirmOrder que orquesta el dominio), adapters/in/http/ (el controlador que recibe la peticion), adapters/out/persistence/ (el repositorio que guarda en la base de datos) y config/ (el punto de arranque).
- Trazamos el recorrido completo de una peticion para dejarlo concreto: POST /orders llega al controlador, el controlador llama al caso de uso ConfirmOrder, el caso de uso ejecuta orden.confirm() (la misma regla que protegimos en el Sprint 3) y despues guarda a traves del puerto OrderRepository, que el adaptador de Postgres implementa.
- Armamos el punto de arranque (composition root): ahi, y solo ahi, se crea el adaptador concreto PostgresOrderRepository y se inyecta en ConfirmOrder, que a su vez se inyecta en el controlador. En ningun lugar del dominio ni de los casos de uso aparece un "new PostgresOrderRepository()".
- Construimos el walking skeleton real: un endpoint GET /health y un POST /orders minimo, corriendo contra una base de datos Postgres real en un contenedor - no solo compila, efectivamente corre y responde.

3. Una decision que nos costo tomar

Discutimos si convenia terminar primero el dominio completo de las seis areas y recien despues conectar la base de datos y el servidor web, o si era mejor empezar por un solo camino que atraviese todas las capas. El primer camino es justo el error que la sesion advierte como "big bang": dominio tres semanas, base de datos dos, web una, y al final nada corre para la demo.

4. Como lo resolvimos

Invertimos el orden: primero el walking skeleton (health + una entidad real persistida) y a partir de ahi crecemos en cortes verticales delgados, integrando desde el primer commit en lugar de dejarlo para el final. Tambien dejamos como regla que ningun archivo dentro de domain/ puede importar el driver de la base de datos - lo revisamos a mano en esta primera version.

5. En que va el proyecto

Cerramos la sesion con el walking skeleton de ordenes-service corriendo de verdad: /health responde y POST /orders persiste una orden en Postgres real dentro de un contenedor. Lo que sigue, en la sesion de planeacion, es definir el contrato de la API y el alcance exacto de MVP 1 sobre este esqueleto.

6. Evidencia

Campo	Detalle
Commit	"Walking skeleton de ordenes-service"
Repositorio	github.com/BrandowC/amaranta-shop-docs